

BookForum — Documentação do Sistema e Especificação da API

Disciplina: Desenvolvimento Web Back-end (BRADWBK) **Domínio (produção):** <https://bookforum.vgrn.cloud>

BookForum é uma rede social de leitura (estilo "Instagram para livros"): os usuários criam conta, publicam *reviews* de livros, curtem e comentam, trocam mensagens privadas em tempo real, favoritam livros, ganham XP por leituras, participam de comunidades e recebem notificações.

1. Integrantes do grupo

Nome	Prontuário
(preencher)	(preencher)
(preencher)	(preencher)

2. Visão geral e arquitetura

Aplicação web full-stack composta por:

- **Backend:** API REST + WebSocket em **Spring Boot 3.5** (Java 17).
- **Frontend:** SPA em **Vue 3 + Vuetify 3** (Vite), empacotável como app mobile via **Capacitor**.
- **Banco de dados:** **MySQL 8** (ORM JPA/Hibernate).
- **Autenticação:** **JWT (HS256)** com senha criptografada em **BCrypt**.
- **Implantação:** **Docker Compose** atrás de um **NGINX** (proxy reverso e balanceador de carga no host) com HTTPS via Certbot.

Fluxo das requisições: a Internet chega via HTTPS ao **NGINX do host** (proxy reverso + TLS, domínio bookforum.vgrn.cloud), que encaminha as rotas `/` para o container **frontend** (NGINX interno, porta 3002→80) e as rotas `/api` e `/ws` para o(s) container(s) **backend** (Spring Boot, porta 8086). O backend acessa o **MySQL 8** (container `db`, porta 3306, volume persistente `mysql_data`). A comunicação container → container usa a rede interna do Docker (<http://backend:8086>), independente das portas publicadas no host.

Organização do repositório: duas aplicações — `backend/` (Spring Boot, com os pacotes `config`, `controller`, `service`, `repository`, `model`, `dto`) e `frontend/` (Vue, com as pastas `views`, `components`, `services`, `composable`, `router`, `plugins`) — além de `docker-compose.yml`, `docker-compose.lb.yml` (balanceamento) e a pasta `nginx/` (configuração do balanceador).

3. Stack tecnológica

Camada	Tecnologias
Backend	Spring Boot 3.5.7, Spring Web, Spring Data JPA, Spring Security, Spring WebSocket, JWT, Java 17

Banco	MySQL 8 (driver mysql-connector-j)
ORM	JPA / Hibernate
Autenticação	JWT (HS256, biblioteca JJWT) + BCrypt
Frontend	Vue 3.5, Vuetify 3.10, Vue Router 4, Pinia, Axios, Vite 7
Tempo real	STOMP sobre SockJS (@stomp/stompjs , sockjs-client)
Mobile	Capacitor (Android/iOS)
Build/infra	Gradle, Vite, Docker, Docker Compose, NGINX

4. Entidades e relacionamentos

Quatro entidades centrais (há outras complementares no sistema):

Entidade	Descrição
User	Usuário da plataforma (nome, e-mail, senha BCrypt, avatar, bio, papel).
Post	<i>Review</i> de um livro publicada por um usuário (capa, texto, curtidas, comentários).
Book	Livro cadastrado (título, autor, descrição, capa, dono).
Message	Mensagem privada trocada entre dois usuários.

Relacionamentos exigidos:

- **Um-para-Muitos (1:N):** [User](#) → [Post](#). Um usuário possui vários posts; cada post pertence a um único autor ([Post.user](#), [@ManyToOne](#)). (*Também há [Post](#) → [PostComment](#) e [Book](#) → [Topic](#).*)
- **Muitos-para-Muitos (N:M):** [User](#) ↔ [Book](#) (livros favoritos), via tabela de junção [user_favorite_books](#) ([User.favoriteBooks](#), [@ManyToMany](#)).

Modelo de dados completo (entidades JPA)

Entidade	Campos principais
User	id , username , email , password (BCrypt), avatarUrl , avatarData (LONGBLOB), avatarContentType , bio , role (USER/ADMIN), favoriteBooks
Book	id , title , author , description , coverImageUrl , owner , available
RegisterBook	dados de cadastro/registro de livro pelo usuário
Post	autor, livro, texto, capa, curtidas, comentários
PostComment	autor, conteúdo, post
PostLike	usuário, post
Message	senderId , receiverId , content , createdAt
Notification	userId , type , title , body , read , createdAt

Community	comunidade de leitores
Topic	tópico vinculado a um livro
Donation	doação de livro
Activity / UserActivity	atividade criada por admin e sua atribuição/progresso
UserXP	gamificação: <code>level</code> , <code>totalBooksRead</code> , <code>streak</code> , <code>xp</code>

Foto de perfil: a imagem é guardada **no banco** (`User.avatarData` como `LONGBLOB` + `avatarContentType`); o campo `avatarUrl` aponta para o endpoint que serve esses bytes (`/api/users/{id}/avatar?v=<timestamp>`). Assim a foto **sobrevive a rebuilds** do container (o disco do container é efêmero; o MySQL persiste no volume `mysql_data`).

A criação do schema é automática (`spring.jpa.hibernate.ddl-auto=update`).

5. Autenticação (JWT)

A API usa **autenticação stateless com JSON Web Token (JWT, HS256)** e senha criptografada com **BCrypt**. Toda a API exige token, exceto as rotas públicas.

Como funciona

- O **token** é gerado no `register/login` (`JwtService`, segredo em `JWT_SECRET`, expiração em `JWT_EXPIRATION_MS`, padrão 24h) e carrega `subject` (e-mail), `userId`, `username` e `role`.
- O cliente envia o token em **todas** as requisições no header `Authorization: Bearer <token>` (interceptor do Axios em `services/api.ts`).
- O `JwtAuthFilter` valida o token e popula o `SecurityContext`. Token inválido/expirado → **401** (o frontend desloga e volta ao login).
- O **logout** invalida o token no servidor (`TokenBlacklist`), impedindo reuso até a expiração.

Rotas públicas (sem token)

`POST /api/auth/**`, `GET /api/users/{id}/avatar`, `GET /uploads/**`, `/ws` (WebSocket) e as requisições `OPTIONS` (pré-flight CORS).

Papéis

- `USER` (padrão) e `ADMIN`.
- O **superusuário** é promovido a `ADMIN` automaticamente no login.
- Promoção de outros usuários: `POST /api/users/{id}/promote` (só o superusuário).

6. Especificação REST da API

Formato: **JSON** (exceto o upload de avatar, que é `multipart/form-data`). Datas em ISO-8601. **Toda rota protegida sem token válido retorna 401 Unauthorized** — esse **401** está implícito nas tabelas (exceto nas rotas públicas).

6.1. Autenticação — /api/auth

Verbo	Path	Body de Requisição	Body de Retorno	Sucesso	Erro
POST	/api/auth/register	{ "username": "ana", "email": "ana@x.com", "password": "123" }	{ "token": "...", "id": 1, "username": "ana", "email": "ana@x.com", "avatarUrl": null, "role": "USER", "bio": null }	201	400, 409
POST	/api/auth/login	{ "email": "ana@x.com", "password": "123" }	{ "token": "...", "id": 1, "username": "ana", "role": "USER", ... }	200	401, 404
POST	/api/auth/logout	— (header Authorization: Bearer <token>)	"Logout efetuado"	200	—

6.2. Usuários — /api/users (entidade User, CRUD completo)

Verbo	Path	Body de Requisição	Body de Retorno	Sucesso	Erro
GET	/api/users	—	[{ "id": 1, "username": "ana", "email": "...", "avatarUrl": "...", "role": "USER", "bio": "..." }]	200	401
GET	/api/users/{id}	—	{ "id": 1, "username": "ana", ... }	200	404
GET	/api/users/email/{email}	—	{ "id": 1, "username": "ana", ... }	200	404
PUT	/api/users/{id}	{ "username": "ana2", "email": "...", "bio": "...", "password": "(opcional)" }	{ "id": 1, "username": "ana2", ... }	200	404
DELETE	/api/users/{id}	—	—	204	404
POST	/api/users/{id}/avatar	multipart/form-data campo file	{ "id": 1, "avatarUrl": "/api/users/1/avatar?v=...", ... }	200	400, 404
GET	/api/users/{id}/avatar (pública)	—	bytes da imagem (image/*)	200	404
POST	/api/users/{id}/promote?requesterId=	—	{ "id": 1, "role": "ADMIN", ... }	200	403, 404

6.3. Posts / Reviews — /api/posts (entidade Post, CRUD completo)

Verbo	Path	Body de Requisição	Body de Retorno	Sucesso	Erro
-------	------	--------------------	-----------------	---------	------

GET	/api/posts/feed?userId={id}		[{ "id": 1, "userId": 2, "username": "ana", "bookTitle": "...", "content": "...", "likesCount": 3, "likedByCurrentUser": true, "comments": [...] }]	200	401
GET	/api/posts/user/{userId}?currentUserId={id}		[{ ...PostDTO }]	200	401
POST	/api/posts	{ "userId": 2, "bookTitle": "1984", "bookAuthor": "Orwell", "coverImageUrl": "...", "content": "Ótimo!" }	{ ...PostDTO }	200	401
POST	/api/posts/{id}/like?userId={id}		{ ...PostDTO }	200	401
POST	/api/posts/{id}/comment	{ "userId": 2, "content": "Concordo!" }	{ ...PostDTO }	200	401
DELETE	/api/posts/{id}?userId={id}		—	204	401

6.4. Mensagens — /api/messages (entidade Message)

Verbo	Path	Body de Requisição	Body de Retorno	Sucesso	Erro
POST	/api/messages	{ "senderId": 1, "receiverId": 2, "content": "oi" }	{ "id": 5, "senderId": 1, "receiverId": 2, "content": "oi", "createdAt": "..." }	201	400
GET	/api/messages/conversation/{a}/{b}		[{ ...MessageDTO }]	200	401
GET	/api/messages/for/{userId}		[{ ...MessageDTO }]	200	401

6.5. Livros — /api/books (entidade Book)

Verbo	Path	Body de Requisição	Body de Retorno	Sucesso	Erro
GET	/api/books	—	[{ "id": 1, "title": "1984", "author": "Orwell", "available": true }]	200	401
POST	/api/books	{ "title": "1984", "author": "Orwell", "description": "...", "coverImageUrl": "..." }	{ "id": 1, "title": "1984", ... }	200	401
GET	/api/books/users/{userId}/books		[{ ...Book }]	200	401

Relacionamento N:M (favoritos): persistido na tabela de junção `user_favorite_books` (entidades `User` ↔ `Book`).

6.6. Demais recursos

Recurso	Base	Operações
Notificações	<code>/api/notifications</code>	<code>GET /for/{userId}</code> , <code>PATCH /{id}/read</code>
XP / Gamificação	<code>/api/xp</code>	<code>GET /{userId}</code> , <code>POST /{userId}/book-read</code> , <code>GET /leaderboard</code>
Atividades	<code>/api/activities</code>	CRUD de admin + <code>GET /user/{userId}</code> , <code>POST /user/{id}/complete</code>
Registro de livros	<code>/api/register-books</code>	<code>GET</code> , <code>GET /search</code> , <code>POST</code>
Tópicos	<code>/api/topics</code>	<code>GET /book/{bookId}</code>
Comunidades	<code>/api/communities</code>	<code>GET</code> , <code>POST</code>
Doações	<code>/api/donations</code>	<code>GET</code> , <code>POST</code>

7. WebSocket (chat em tempo real)

Configurado em `WebSocketConfig` (STOMP + SockJS):

- **Endpoint de conexão:** `/ws` (SockJS, origens *).
- **Broker simples:** tópicos `/topic` e filas `/queue`.
- **Prefixo de aplicação:** `/app` (mensagens do cliente para `@MessageMapping`).
- **Destino de usuário:** `/user` (`convertAndSendToUser`).

O frontend usa `@stomp/stompjs` + `sockjs-client` (`services/socket.ts`) para receber as mensagens da conversa em tempo real.

8. Frontend

Rotas (`router/index.ts`)

Caminho	Tela	Observação
<code>/</code>	<code>home.vue</code>	Landing + login/cadastro
<code>/feed</code>	<code>FeedView.vue</code>	Feed principal
<code>/explore</code>	<code>ExploreView.vue</code>	Explorar
<code>/create</code>	<code>CreatePostView.vue</code>	Criar post
<code>/profile</code>	<code>ProfileView.vue</code>	Perfil (posts, XP, bio)
<code>/leaderboard</code>	<code>LeaderboardView.vue</code>	Ranking

/message	message.vue	Conversas
/myBooks	myBooks.vue	Biblioteca
/activities	ActivitiesView.vue	Atividades
/notifications	NotificationsView.vue	Notificações
/admin	AdminView.vue	Painel admin (requer ADMIN)

As rotas internas exigem autenticação (`meta.requiresAuth`); `/admin` exige papel `ADMIN` (`meta.requiresAdmin`).

Navegação (AppShell.vue)

Layout estilo Instagram:

- **Barra superior:** logo "BookForum", alternância de tema, **Notificações** (com badge) e atalho de admin para `ADMIN`.
- **Barra inferior:** Feed · Explorar · Criar · **Mensagens** (com badge) · Perfil.

As contagens (badges) são atualizadas a cada 30s: notificações do tipo `message` alimentam o badge de Mensagens; as demais, o de Notificações.

Serviços (services/*.ts)

Cada recurso tem seu cliente Axios: `api.ts` (instância base com interceptor de token), `apiLogin`, `apiRegisterUsers`, `apiPosts`, `apiMessages`, `apiNotifications`, `apiXP`, `apiActivities`, `apiBooks`, `apiRegisterBook`, `apiTopics`, `apiCommunity` e `socket.ts` (WebSocket).

9. Funcionalidades principais

- **Feed de reviews:** posts com capa, texto, curtidas e comentários.
- **Perfil:** avatar (no banco), bio, estatísticas (posts, livros lidos, nível), conquistas (chips de XP) e grade de posts.
- **Mensagens:** chat privado entre usuários, em tempo real via WebSocket.
- **Notificações:** avisos diversos; o tipo `message` é contabilizado à parte.
- **Gamificação (XP):** nível, total de livros lidos, *streak* de dias e ranking.
- **Atividades:** tarefas criadas e atribuídas por admins, com progresso e conclusão.
- **Comunidades, Tópicos, Doações, Biblioteca:** recursos complementares.
- **Tema claro/escuro e app mobile** (Capacitor).

10. Implantação (Docker + NGINX)

Containers (docker-compose.yml)

Serviço	Porta (host → container)	Volume
<code>db</code> (MySQL 8)	3306 → 3306	<code>mysql_data</code> (persistente)

backend (Spring Boot)	8086 → 8086	—
frontend (NGINX)	3002 → 80	—

Atenção à porta do backend: o app escuta na **8086** dentro do container, então o mapeamento publicado **deve** ser **8086:8086**. Um mapeamento errado (ex.: **8086:8080**) faz o NGINX do host bater em uma porta morta e retornar **502 Bad Gateway**.

Deploy na VPS

```
git pull
docker compose up -d --build # rebuild é obrigatório p/ refletir mudanças de código
docker compose ps            # conferir STATUS e PORTS
```

Balanceamento de carga com NGINX (round-robin)

Uma 2ª instância do backend (opt-in) distribui a carga via NGINX:

- `docker-compose.lb.yml` → adiciona `backend2` (porta **8087** → **8086**).
- `nginx/bookforum-loadbalancer.conf` → bloco `upstream bookforum_backend` com as duas instâncias (**127.0.0.1:8086** e **127.0.0.1:8087**); o `/api` passa a usar `proxy_pass http://bookforum_backend`.

Subir com balanceamento e evidenciar (o header `X-Upstream` alterna entre `:8086` e `:8087`):

```
docker compose -f docker-compose.yml -f docker-compose.lb.yml up -d --build
curl -s -o /dev/null -D - https://bookforum.vgrn.cloud/api/users | grep -i x-upstream
```

Para o teste **sem** balanceamento, basta deixar apenas uma instância no `upstream` (comentar o `server 127.0.0.1:8087`;) e recarregar o NGINX.

11. Variáveis de ambiente

Variável	Padrão	Onde
<code>PORT</code>	8086	backend (porta do servidor)
<code>SPRING_DATASOURCE_URL</code>	<code>jdbc:mysql://localhost:3306/bookgram</code>	backend
<code>SPRING_DATASOURCE_USERNAME</code>	<code>bookgram_user</code>	backend
<code>SPRING_DATASOURCE_PASSWORD</code>	1234	backend
<code>ALLOWED_ORIGINS</code>	origens locais + <code>*.vercel.app</code>	backend (CORS)
<code>JWT_SECRET</code>	segredo padrão (trocar em produção)	backend (assinatura do JWT)
<code>JWT_EXPIRATION_MS</code>	86400000 (24h)	backend (validade do token)
<code>VITE_API_URL</code>	—	frontend (base da API)

No `docker-compose.yml`, o backend aponta para o banco via `jdbc:mysql://db:3306/bookgram` (nome do serviço Docker `db`).

12. Desenvolvimento local

Backend — `cd backend && ./gradlew bootRun` (sobe em `http://localhost:8086`, precisa de MySQL).

Frontend — `cd frontend && npm install && npm run dev` (Vite em `http://localhost:5173`).

Tudo via Docker — `docker compose up -d --build` (frontend em `:3002`, backend em `:8086`, db em `:3306`).